Unfortunately, permutation invariance does not hold column-wise, i.e., $\mathrm{T}(\boldsymbol{Z}\boldsymbol{P}_C) \neq \mathrm{T}(\boldsymbol{Z})\boldsymbol{P}_C$; rather, we could achieve $\mathrm{T}_{(C)}(\boldsymbol{Z}\boldsymbol{P}_C) = \mathrm{T}(\boldsymbol{Z})\boldsymbol{P}_C$ by adjusting the weights in $\mathrm{T}$ according to

$$\boldsymbol{W}_{(C)} = \boldsymbol{P}_C^{-1}\boldsymbol{W}\boldsymbol{P}_C. \tag{6}$$

Formally, the property holds as

**Theorem 4.4** (**Column Permutation Forward Equivariance**). *Stacked Transformer encoder and decoder is permutation equivariant w.r.t. column permutations, in forward propagation, i.e.,* $\mathrm{T}_{(C)}(\boldsymbol{Z}\boldsymbol{P}_C) = \mathrm{T}(\boldsymbol{Z})\boldsymbol{P}_C$ *for any permutation matrix* $\boldsymbol{P}_C \in \mathbb{R}^{d\times d}$*, where weights in* $\mathrm{T}_{(C)}$ *are weights in* $\mathrm{T}$ *permuted by Eq. 6.*

With Thm. 4.4, we can easily derive that $\hat{\boldsymbol{Y}}_{(C)}$ in Eq. 5 is equal to $\hat{\boldsymbol{Y}}$ in Eq. 1. This property is depicted in Fig. 2(b). The case bears resemblance to homomorphic encryption, wherein the input $\boldsymbol{Z}$ and backbone weights $\boldsymbol{W}$ are 'encrypted' by $\boldsymbol{P}_C$, and the output is 'decrypted' using $\boldsymbol{P}_C^{-1}$. Although shuffling does not offer the same level of security compared to homomorphic encryption, it disguises the original inputs and models to some extent. Only those who possess the key $\boldsymbol{P}_C$ can obtain the correct output and effectively use the model. Without the key, however, it is hard to guess the correct output, or take advantage of the model (see Sec. 5).

Likewise, in the corresponding backprop of column-permuted forwarding (Eq. 5), the property holds as:

**Theorem 4.5** (**Column Permutation Backward Equivariance**). *In backward propagation, gradients of Transformer parameters in the natural setting (Eq. 1) and the permuted setting (Eq. 5) are correlated by the following equation:*

$$\frac{\partial l}{\partial \boldsymbol{W}_{(C)}} = \boldsymbol{P}_C^{-1}\frac{\partial l}{\partial \boldsymbol{W}}\boldsymbol{P}_C \tag{7}$$

*where* $l$ *is the loss function.*

With simple induction, we have:

**Corollary 4.6.** *The weights of Transformer learned in the natural setting (Eq. 1),* $\mathrm{T}$*, and that learned in the permuted setting (Eq. 5),* $\mathrm{T}_{(C)}$*, are correlated by* $\boldsymbol{W}_{(C)} = \boldsymbol{P}_C^{-1}\boldsymbol{W}\boldsymbol{P}_C$*, given the same randomly initialized weights.*

## 4.2. General Permutation Equivariant Networks

In this section, we show that the row and column permutation equivariance in the forward and backward propagations can be generalized to a broader class of permutation equivariant networks, apart from Transformers. Here permutation generally refers to row and column permutation:

$$\boldsymbol{Z}_{(P)} = \boldsymbol{P}_R F_1(\boldsymbol{X})\boldsymbol{P}_C,\ \hat{\boldsymbol{Y}}_{(P)} = F_2(\boldsymbol{P}_R^{-1} f_{(P)}(\boldsymbol{Z}_{(P)})\boldsymbol{P}_C^{-1}), \tag{8}$$

where $f_{(P)}$ is the Transformer backbone $f$ has its weight permuted by Eq. 6.

We show the conclusion in three steps: 1) a majority of common neural network operators satisfy forward permutation equivariance; 2) if an operator is permutation-equivariant in forwarding, it must be permutation-equivariant in the backward propagation; 3) a network composed by the aforementioned operators are both forward and backward permutation-equivariant.

We provide proofs for 1) in Appendix 9 covering a wide variety of operators, such as linear projection, attention, norms, element-wise operators (shortcut skip, Hadamard product, activation, etc.), and softmax. The exception is the type of operators working on sliding windows, e.g., convolutional. For 2) and 3), we will prove the following theorem:

**Theorem 4.7** (General Permutation Equivalent Networks). *If* $f$ *is composed by* $f_N \circ f_{N-1} \circ \cdots \circ f_1$ *where* $f_1, \ldots, f_N$ *that are permutation-equivariant in the forward propagation, and each contains weights (if any) as linear arguments, i.e.,* $f = f_N(\cdots f_2(f_1(\boldsymbol{Z}\boldsymbol{W}_1^\top)\boldsymbol{W}_2^\top)\cdots\boldsymbol{W}_N^\top)$*,* $f$ *is permutation-equivariant in the backward propagation, i.e., the weights* $\boldsymbol{W}$ *in* $f$ *are associated with those of* $f_{(P)}$ *by* $\boldsymbol{W}_{(P)} = \boldsymbol{P}_C^{-1}\boldsymbol{W}\boldsymbol{P}_C$*.*

To prove Thm. 4.7, we first prove the following lemma:

**Lemma 4.8.** *If* $f$ *is composed by permutation-equivariant operators as in Thm. 4.7, the derivative of* $f$ *with respect to feature* $\boldsymbol{Z}$ *in the natural (Eq. 1) and in the permuted (Eq. 8) settings are correlated by*

$$\frac{\partial l}{\partial \boldsymbol{Z}_{(P)}} = \boldsymbol{P}_R\frac{\partial l}{\partial \boldsymbol{Z}}\boldsymbol{P}_C. \tag{9}$$

*Proof.* It is obvious that in forwarding, the combination of permutation-equivariant operators remains to be permutation-equivariant since one can perform induction on the simple case of $f_{1(P)}(f_{2(P)}(\boldsymbol{P}_R\boldsymbol{Z}\boldsymbol{P}_C)) = f_{1(P)}(\boldsymbol{P}_R f_2(\boldsymbol{Z})\boldsymbol{P}_C) = \boldsymbol{P}_R f_1(f_2(\boldsymbol{Z}))\boldsymbol{P}_C$. Hence through all layers in $f$, it holds that

$$f_{(P)}(\boldsymbol{Z}_{(P)}) = \boldsymbol{P}_R f(\boldsymbol{Z})\boldsymbol{P}_C. \tag{10}$$

By Eq. 8 and Eq. 10, the forward propagation feeds $f(\boldsymbol{Z})$ into $F_2$, which is equivalent to the vanilla forwarding, and hence the losses are the same.

Now consider the backward propagation. As the loss $l$ is invariant by permutation, we differentiate $l$ through $\boldsymbol{Z}_{i(P)}$ which is the intermediate output of layer $i$ of $f_{(P)}$. By the forward permutation equivariance, we know that it is associated with $\boldsymbol{Z}_i$, the output of layer $i$ at $f$, as $\boldsymbol{Z}_{i(P)} = \boldsymbol{P}_R\boldsymbol{Z}_i\boldsymbol{P}_C$. Hence we have $\mathrm{d}\boldsymbol{Z}_{i(P)} = \boldsymbol{P}_R\mathrm{d}\boldsymbol{Z}_i\boldsymbol{P}_C$. Therefore, the derivative of $l$ with respect to $\boldsymbol{Z}_{i(P)}$ for any layer $i$

in the permuted setting is

$$\mathrm{d}l \triangleq \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{Z}_{i(P)}}^\top \mathrm{d}\boldsymbol{Z}_{i(P)}) = \mathrm{tr}(\frac{\partial l}{\partial \boldsymbol{Z}_{i(P)}}^\top \boldsymbol{P}_R \mathrm{d}\boldsymbol{Z}_i \boldsymbol{P}_C)$$
$$= \mathrm{tr}(\boldsymbol{P}_C \frac{\partial l}{\partial \boldsymbol{Z}_{i(P)}}^\top \boldsymbol{P}_R \mathrm{d}\boldsymbol{Z}_i) = \mathrm{tr}((\boldsymbol{P}_R^\top \frac{\partial l}{\partial \boldsymbol{Z}_{i(P)}} \boldsymbol{P}_C^\top)^\top \mathrm{d}\boldsymbol{Z}_i).$$

The last equality suggests $\frac{\partial l}{\partial \boldsymbol{Z}_i} = \boldsymbol{P}_R^\top \frac{\partial l}{\partial \boldsymbol{Z}_{i(P)}} \boldsymbol{P}_C^\top$ according to Thm. 6 of [14]. Since $\boldsymbol{P}_R^\top = \boldsymbol{P}_R^{-1}$ and $\boldsymbol{P}_C^\top = \boldsymbol{P}_C^{-1}$, Eq. 9 holds completing the proof. □

Then we prove Thm. 4.7.

*Proof.* Without causing any confusion, let $\boldsymbol{Z}_i$ denote the output of $f_i(f_{i-1}(\cdots f_1(\boldsymbol{Z}\boldsymbol{W}_1^\top)\cdots \boldsymbol{W}_{i-1}^\top)\boldsymbol{W}_i^\top)$. We differentiate the loss through $\boldsymbol{Z}_i$ by

$$\mathrm{d}l \triangleq \mathrm{tr}((\frac{\partial l}{\partial \boldsymbol{Z}_i})^\top \mathrm{d}\boldsymbol{Z}_i) = \mathrm{tr}((\frac{\partial l}{\partial \boldsymbol{Z}_i})^\top \mathrm{d}(\boldsymbol{Z}_{i-1}\cdot \boldsymbol{W}_i^\top))$$
$$= \mathrm{tr}((\frac{\partial l}{\partial \boldsymbol{Z}_i})^\top \boldsymbol{Z}_{i-1}\mathrm{d}\boldsymbol{W}_i^\top) = \mathrm{tr}((\frac{\partial l}{\partial \boldsymbol{Z}_i}^\top \boldsymbol{Z}_{i-1})^\top \mathrm{d}\boldsymbol{W}_i).$$

Hence the gradient of $\boldsymbol{W}_i$ is $\frac{\partial l}{\partial \boldsymbol{W}_i} = \frac{\partial l}{\partial \boldsymbol{Z}_i}^\top \boldsymbol{Z}_{i-1}$. Similarly, the gradient of $\boldsymbol{W}_{i(P)}$ is as follows:

$$\frac{\partial l}{\partial \boldsymbol{W}_{i(P)}} = \frac{\partial l}{\partial \boldsymbol{Z}_{i(P)}}^\top \boldsymbol{Z}_{i-1(P)} = \boldsymbol{P}_C^\top \frac{\partial l}{\partial \boldsymbol{Z}_i}^\top \boldsymbol{P}_R^\top \boldsymbol{P}_R \boldsymbol{Z}_{i-1}\boldsymbol{P}_C$$
$$= \boldsymbol{P}_C^\top \frac{\partial l}{\partial \boldsymbol{Z}_i}^\top \boldsymbol{Z}_{i-1}\boldsymbol{P}_C = \boldsymbol{P}_C^{-1}\frac{\partial l}{\partial \boldsymbol{W}_i}\boldsymbol{P}_C. \tag{11}$$

The second equality holds due to permutation equivalence of $\boldsymbol{Z}_{i-1}$ and Lem. 4.8. Following a similar argument to Corollary 4.6, we have $\boldsymbol{W}_{i(P)} = \boldsymbol{P}_C^{-1}\boldsymbol{W}_i\boldsymbol{P}_C$ for any layer $i$, given the same randomly initialized weights in the natural and permuted settings. Thereby the backward permutation equivariance holds. Proof completes. □

It should be noted that the pair of arguments — permutation forward and backward equivariance — seem to be circular, i.e., the backward equivariance holds on the condition of the forward equivariance while the latter holds at $\boldsymbol{W}_{(P)} = \boldsymbol{P}_C^{-1}\boldsymbol{W}\boldsymbol{P}_C$. But one can break such a circle by permuting (the inputs and) the initial weights by Eq. 6 in the first place. Then the forward equivariance holds and one can train the model with Eq. 5. Eventually, the backward equivariance can be derived. In the case of training from scratch, it does not matter if the initial weights are permuted since they are all random.

**This shows why the permutation-invariance in Set Transformer [17] is a pseudo-invariance.** The proposed permutation-invariance head, PMA [17], only achieves forward permutation invariance. However, input of PMA does not follow Lem. 4.8, but rather $\frac{\partial l}{\partial \boldsymbol{Z}_{(P)}} = \frac{\partial l}{\partial \boldsymbol{Z}}$ which destroys the permutation equivariant structure of Eq. 11: instead of getting $\frac{\partial l}{\partial \boldsymbol{W}_{i(P)}} = \frac{\partial l}{\partial \boldsymbol{Z}_i}^\top \boldsymbol{P}_R^\top \boldsymbol{P}_R \boldsymbol{Z}_{i-1}$, they obtain $\frac{\partial l}{\partial \boldsymbol{W}_{i(P)}} = \frac{\partial l}{\partial \boldsymbol{Z}_i}^\top \boldsymbol{P}_R \boldsymbol{Z}_{i-1}$ that the gradients are not aligned anymore.

### 4.3. Other Components

Notably, the weights in Layer Norm ($\gamma$), bias ($b$) and MLP defined in the form of $\mathrm{MLP}(\boldsymbol{X}) = \sigma(\boldsymbol{X}\boldsymbol{W}_1^\top)\boldsymbol{W}_2^\top$ do not follow Eq. 6 for permutation equivariance. But the intuition is the same: pure token shuffling (row permutation) would not affect these operators; column permutation should be corrected by parameters permutation:

$$\gamma_{(C)} = \gamma\boldsymbol{P}_C,\ b_{(C)} = b\boldsymbol{P}_C, \tag{12}$$
$$\boldsymbol{W}_{1(C)} = \boldsymbol{W}_1\boldsymbol{P}_C,\ \boldsymbol{W}_{2(C)} = \boldsymbol{P}_C^{-1}\boldsymbol{W}_2. \tag{13}$$

The proof can be found in Appendix 9 and 10. Parameters in $F_1$, including position embedding, are also not influenced. Proofs can be found in Appendix 10.3.

It is worth noting that the position embedding in $F_1(\cdot)$ is not affected by permutation in both forward and backpropagation since it is added before permutation. The proof in Appendix 10.3 shows that parameters outside of the permutation-inversion scheme are not affected.

## 5. Experiments

In this section, we provide empirical evidence to support our theoretical findings. As a proof-of-concept, we conduct a series of experiments to demonstrate the potential applications of these properties in real-world scenarios.

### 5.1. Setup

Our implementation is built on `Pytorch` and `Torchvision`. We validate our theorems by a range of models and tasks including: the `timm` [1] version of pre-trained ViT-Base for image classification, the official version of ViT-Adapter [5] for semantic segmentation (pre-trained with DeiT, using UperNet), huggingface's pre-trained Bert and GPT2 for text classification, and GPT2 for text generation. The image tasks are chosen as 10-label classification on Cifar10 [16] consisting of 60,000 natural images, semantic segmentation on ADE20k [38, 39] containing 25,210 images with pixel-level annotations. For text classification, we use IMDB [23] dataset with 50,000 movie reviews for fine-tuning a sentimental classifier, and a natural language inference dataset SNLI[2] with over 500k sentences. For text generation, we use the GPT2 trained by huggingface on WebText dataset [27] for zero-shot generation on WikiText2 dataset [25].

[1] https://github.com/rwightman/pytorch-image-models
[2] https://nlp.stanford.edu/projects/snli/

Table 2. Test results (%) of ViT-Base for image classification and ViT-Adapter for segmentation. 'Col.' means column and 'P.' stands for permutation.

| | ViT-Base | ViT-Adapter | | |
|---|---|---|---|---|
| | Acc | aAc | mIoU | mAcc |
| Test w/o P. | 97.65 | 83.12 | 48.75 | 60.09 |
| Test w/ Row P. | 97.65 | 83.12 | 48.75 | 60.09 |
| Test w/ Col. P. | 97.65 | 83.12 | 48.75 | 60.09 |

Table 3. Test accuracy (%) of ViT-Base for image classification and Bert for text classification. Models are trained by Eq. 1 and Eq. 2.'Col.' means column and 'P.' stands for permutation.

| | ViT-Base | Bert (Small) |
|---|---|---|
| Train w/o P. | 97.75 | 76.4 |
| Train w/ Row P. | 97.73 | 76.5 |
| Train w/ Col. P. | 97.94 | - |

Table 4. Test accuracy (%) of Bert and GPT2 for text classification, and perplexity of GPT2(G) for text generation.

| | Bert | GPT2 | GPT2(G) |
|---|---|---|---|
| Test w/o P. | 94.00 | 94.03 | 48.55 |
| Test w/ Col. P. | 94.00 | 94.03 | 48.55 |

All Transformer backbones are of base size, characterized by a token dimension of 768, 12 heads, and 12 layers. The evaluation metrics are test accuracy for classification tasks, mIoU, aAcc, and mAcc for semantic segmentation, and perplexity for text generation. More detailed setup can be found in Appendix 11.

## 5.2. Properties Validation

**Row Permutation Forward Equivariance.** We validate Thm. 4.1 by performing inference on 'vit_base_patch16_224' pre-trained timm model, the officially released ViT-Adapter, and a small Bert with 2 layers and input of size $(128, 256)$, since the padding mask in huggingface implemented Bert do not follow the properties. The ViT-Base model and the small Bert are fine-tuned on Cifar10 and SNLI respectively before inference. We feed into the same trained model the permuted (Eq. 2) and normal (Eq. 1) features of test data, respectively.

The results for image tasks are displayed in Tab. 2. It is evident that the test results for ViT-Base and ViT-Adapter, both with and without row permutation, are identical up to two decimal places. Similarly, the output accuracies of Bert are closely shown in Tab. 3.

**Row Permutation Backward Equivariance.** To validate Thm. 4.2, we train the pre-trained ViT-Base and the small Bert by Eq. 1 and Eq. 2, respectively. As shown in Tab. 3, the test results for ViT-Base and Bert with and without row permutation are almost identical, which is consistent with our findings.

Table 5. Test accuracy (%) of Bert and GPT2 for text classification with models trained by Eq. 1 and Eq. 5.

| | Bert | GPT2 |
|---|---|---|
| Train w/o P. | 94.00 | 94.03 |
| Train w/ Col. P. | 93.72 | 93.66 |

**Col. Permutation Forward Equivariance.** To validate Thm. 4.4, we test ViT-Base, ViT-Adapter, Bert, and GPT2 on corresponding test datasets. The language models are fine-tuned on the IMDB training set before testing. We first test the model trained by Eq. 1 to get the results without permutation, and then permute the trained model by Eq. 6 to get $\mathrm{T}_{(C)}$ in Eq. 5. The model is reported under the 'Test w/ Col. P.' category in tables.

The results are displayed in Tab. 2 and Tab. 4. It is evident that the test results for ViT, Bert, and GPT2, both with and without column permutation, are almost identical. Additionally, if we treat the permuted model $\mathrm{T}_{(C)}$ as a normal model and feed normal inputs, the test accuracy falls to that of random guess, i.e., about 10% on Cifar10 and 50% on IMDB, indicating power similar to 'encryption.' On the text generation task, with or without column permutation, the resulting perplexity remains at $48.55$. If we feed the permuted model $\mathrm{T}_{(C)}$ with normal inputs, the perplexity of the output rises to the order of $10^7$, akin to a randomly initialized language model.

**Col. Permutation Backward Equivariance.** To validate Thm. 4.5, we fine-tune pre-trained ViT-Base, Bert, and GPT2 by Eq. 5 with their initial weights permuted by Eq. 6. Note that if the model is trained from scratch, it is unnecessary to permute its initial weights as they are random. As shown in Tab. 3 and Tab. 5, the test results for the three models, trained with and without column permutation, are very close with minor gaps due to randomness. Thus all properties are validated.

## 5.3. Applications

In this section, we showcase the potential applications of permutation equivariance property in a real-world context as a proof-of-concept. First, we apply the property to enhancing privacy-preserving split learning in [15] and [33]. Following that, we demonstrate how the column permutation of weights can serve as a model 'encryption' or 'authorization' tool, which restricts parties without permutation 'key' from utilizing the model for inference or fine-tuning.

**Privacy-Preserving Split Learning.** We implement the split learning framework following [15, 33] on a multi-label classification task on CelebA dataset [21], which comprises 2,022,599 images from 10,177 celebrities. In the split learning setting, the server holds the Transformer backbone $T$